Technical Documentation

AC Cyclic Voltammetry System for SEED

Repository: `dominicmorris002/redpitaya-rodeostat`

Author: Dominic Morris

Language: Python 3.8

Hardware: Red Pitaya (×2) + Rodeostat Potentiostat + Custom PCB as ACDC Bias + 3D Printed Enclosure

GitHub: https://github.com/dominicmorris002/redpitaya-rodeostat

Last updated: April 18, 2026

0 Contents

1 Project Overview **3**
- 1.1 Project Objectives . . . 3
- 1.2 What is AC Cyclic Voltammetry? . . . 3
- 1.3 Hardware Components . . . 4
- 1.4 What the System Produces . . . 4

2 Hardware Setup **4**
- 2.1 Red Pitaya Network Configuration . . . 4
- 2.2 Enabling the SCPI TCP Server . . . 4
- 2.3 Rodeostat USB Connection . . . 5
- 2.4 Full Physical Wiring + System Setup . . . 5
  - 2.4.1 Power Setup . . . 5
  - 2.4.2 SCPI Setup . . . 5
  - 2.4.3 Signal Wiring . . . 5
  - 2.4.4 Startup Sequence . . . 6
- 2.5 Hardware Photos . . . 6

3 Software Setup **6**
- 3.1 Requirements . . . 6
- 3.2 Downloading the Repository . . . 6
- 3.3 Installing Dependencies . . . 7

4 Repository File Guide **7**
- 4.1 DominicSystem Files — Use These . . . 7
- 4.2 A Files — Use These if DominicSystem is not working . . . 8
- 4.3 Utility Files . . . 9
- 4.4 Old / Dev Scripts . . . 9

5 Running the System **9**
- 5.1 Before You Start . . . 10
- 5.2 Set Your IP Addresses . . . 10
- 5.3 Running . . . 10
- 5.4 Output Files . . . 10

6 How It Works **10**

7 Calibration **11**

8 Troubleshooting **11**

9 Quick-Start **12**

10 Experimental Results **13**
- 10.1 Rodeostat vs. Autolab CV Comparison . . . 13
- 10.2 Redpitaya LIA and DC Ramp Monitor vs. Signal Recovery Comparison . . . 13
- 10.3 Full ACCV Output — Dual Red Pitaya . . . 14

11 Known Limitations and Future Improvements **14**
11.1 Current Limitations 14

12 Hardware Specifications **15**
12.1 Future Improvements 16

13 Acknowledgments **16**

1 Project Overview

This project builds an AC Cyclic Voltammetry (ACCV) measurement system out of two Red Pitaya FPGA boards and a Rodeostat open-source potentiostat. A custom PCB was created to bias the AC wave from the Red Pitaya onto the working electrode of the Rodeostat. The device is enclosed in a simple 3D printed enclosure. Everything runs in Python 3.8. The goal was to put together a setup that can do the same job as a Signal Recovery 7280 DSP Lock-In Amplifier paired with an Autolab Potentiostat at a fraction of the cost, using off-the-shelf hardware and open-source systems.

The focus being to create the ACCV system for the Scanning Electrometer for Electrical Double Layer (SEED) platform. The SEED platform is being developed to detected specific miRNA in a liquid biopsy sample, which will hopefully be used in the medical field as a way of detecting heart disease, cancer, and neurological disorders through the quantity of specific miRNA that is present.

One Red Pitaya handles the Lock-In Amplifier (via PyRPL) and generates the AC signal and measures its magnitude and phase. The second Red Pitaya monitors the DC voltage independently to maintain the same sample rate and plot the LIA Vs the DC Ramp. Both start at the exact same moment using a shared sync file, and their data gets merged and plotted when the run is done. The Rodeostat is used as the potentiostat and applies the DC ramp. The Transimpedance Amplifier (TIA) reads the resulting current and is connected to the Red Pitaya LIA.

1.1 Project Objectives

This project had three core objectives, each targeting a specific piece of expensive commercial instrumentation with an open-source equivalent of comparable accuracy.

Objective 1 — Replace the Autolab Potentiostat. The Autolab PGSTAT302N is a high-end commercial potentiostat used widely in electrochemistry research. The goal was to replace it with the Rodeostat, an open-source potentiostat, while maintaining comparable accuracy in the DC voltage ramp and current measurement. This was checked by running identical CV experiments on both instruments and comparing the results (see Section 10).

Objective 2 — Replace the Lock-In Amplifier. The Signal Recovery 7280 DSP Lock-In Amplifier is a precision instrument used to extract the AC impedance response (magnitude R and phase Θ) from a noisy electrochemical signal. The goal was to replace it using the Red Pitaya's FPGA, programmed as a software Lock-In Amplifier via PyRPL, achieving the same measurement capability at a fraction of the cost. The Red Pitaya generates the AC excitation signal and demodulates the response in real time. This was checked by running identical LIA experiments on both instruments and comparing the results (see Section 10).

Objective 3 — Build a Combined AC Cyclic Voltammetry System. The final objective was to integrate both replacements into a single working ACCV system: the Rodeostat applies the DC potential ramp while the Red Pitaya Lock-In superimposes a small sinusoidal AC signal and simultaneously measures the impedance response. The result is a low-cost, open-source instrument capable of performing the same AC CV experiments previously requiring two separate high-cost commercial instruments working in tandem.

1.2 What is AC Cyclic Voltammetry?

Cyclic Voltammetry (CV) is an electrochemical technique where you sweep a voltage back and forth across an electrochemical cell and measure the current response. AC Cyclic Voltammetry adds

a small sinusoidal AC signal on top of that DC sweep. This lets you measure impedance-related quantities specifically AC magnitude (R) and phase angle (Theta) — as a function of DC potential, all in one experiment.

The technique shows up in studies of reaction mechanisms, surface processes, and electrochemical impedance at different bias points.

1.3 Hardware Components

Component	Role	Connection
Red Pitaya #1 (Lock-In)	Software Lock-In Amplifier; generates AC signal; records R and Theta	Ethernet to PC + USB Power
Red Pitaya #2 (DC Monitor)	Oscilloscope/ADC; reads DC potential from Rodeostat output	Ethernet to PC + USB Power
Rodeostat Potentiostat	Applies DC voltage ramp to the cell; saves current data via USB	USB to PC
PC / Laptop	Runs all scripts; coordinates timing; saves and plots data	USB + Ethernet

1.4 What the System Produces

- CSV files for each measurement channel (Lock-In, DC, Rodeostat)
- 7-panel PNG plots saved automatically
- Interactive HTML plots
- A merged combined CSV linking Lock-In and DC data by index
- Cycle-averaged ACCV plots from post-processing

2 Hardware Setup

2.1 Red Pitaya Network Configuration

Both Red Pitayas communicate over Ethernet. You need to know each board's IP address before running anything. This can be found on the device itself and accessed by typing it into your browser.

1. Connect each Red Pitaya to your local network using a standard Ethernet cable.
2. Power on each board using the micro-USB supply.
3. Find the Red Pitaya address and access each Red Pitaya website.
4. Confirm you can reach them from your PC:

```
rp-f073ce.local    # replace with your Red Pitaya's actual address
```

2.2 Enabling the SCPI TCP Server

The Red Pitaya talks to Python over a TCP SCPI connection. This server has to be started manually through the web interface every time the board reboots, unless configured to autostart. Look for **SCPI** in the **Development** tab.

1. Open a browser and go to the Red Pitaya's address (e.g., `rp-f073ce.local`).
2. You will see the web dashboard with a grid of application tiles.

3. Find the tile labeled **SCPI** under Development.
4. Click **TCP**, then click **Run**.
5. A confirmation will appear saying the SCPI server is running on port 5000.

> **Note:** If the SCPI connection fails when running scripts, go back to the web interface and restart the server. You do not need to reboot the board everytime.

2.3 Rodeostat USB Connection

1. Connect the Rodeostat to your PC via USB.
2. On Windows, check Device Manager to find the COM port (e.g., COM3).
3. Update the port variable in `DominicSystemARodeostat.py` if it doesn't match.

2.4 Full Physical Wiring + System Setup

This is the lab wiring configuration.

2.4.1 Power Setup

- Plug both Red Pitayas into USB power
- Connect both via Ethernet to same router
- Wait for boot LEDs to turn on

2.4.2 SCPI Setup

On BOTH Red Pitayas:

- Open web UI
- Go to Development → SCPI
- Enable TCP mode
- Press RUN (port 5000)

2.4.3 Signal Wiring

Red Pitaya #1 (AC + Lock-In)

- OUT1 → AC bias injection PCB
- TIA output → IN1 (Lock-In input)

Red Pitaya #2 (DC monitoring)

- Rodeostat VOUT → IN1

Rodeostat connections

- WE → working electrode
- RE → reference electrode
- CE → counter electrode

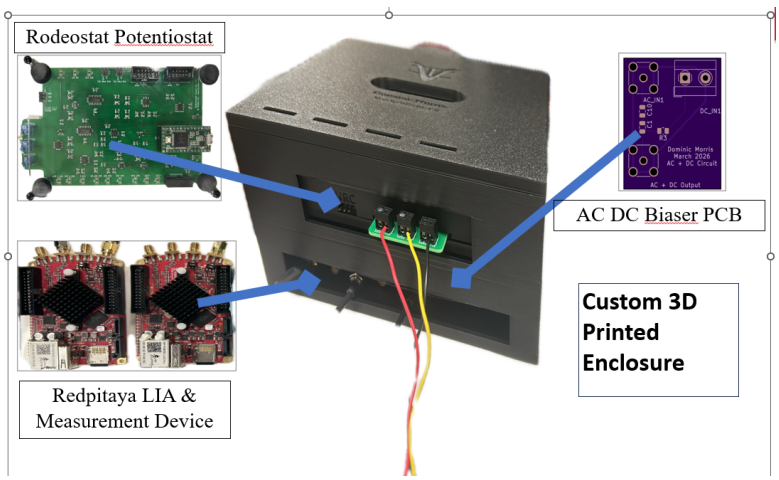
> **IMPORTANT:** Set Red Pitaya input range correctly ($\pm 1\,\mathrm{V}$ or $\pm 20\,\mathrm{V}$ jumper). Wrong setting causes clipping. Effects how precise/accurate the measurement is. This is usually set to $\pm 1\,\mathrm{V}$.

2.4.4 Startup Sequence

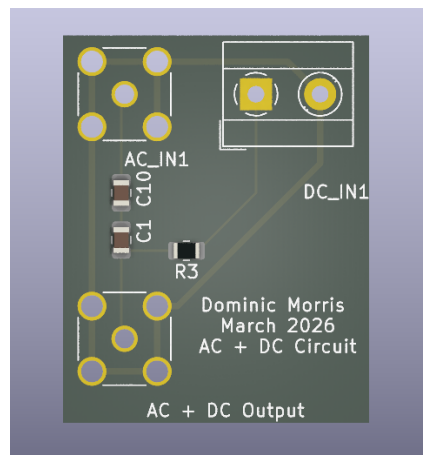
1. Power everything ON
2. Connect Ethernet
3. Enable SCPI on both Red Pitayas
4. Plug in Rodeostat USB
5. Activate Python venv
6. Run script
7. Press Enter (sync)
8. Press Enter again (start)

2.5 Hardware Photos

The 3D-printed enclosure CAD files and the AC+DC Biaser PCB design files are both available in the GitHub repository at `https://github.com/dominicmorris002/redpitaya-rodeostat`.



(a) Assembled system showing the custom 3D-printed enclosure with the Rodeostat, dual Red Pitaya boards, and AC+DC Biaser PCB.



(b) AC+DC Biaser PCB layout (KiCad). Designed by Dominic Morris, March 2026.

Figure 1: Custom hardware: enclosure and PCB.

3 Software Setup

3.1 Requirements

- Python 3.8.10 (use this exact version, PyRPL has issues with newer Python)
- PyCharm or any terminal/IDE
- pip

3.2 Downloading the Repository

Go to `https://github.com/dominicmorris002/redpitaya-rodeostat`, click the green **Code** button, then **Download ZIP**, and extract it wherever you want. The files you want are labeled with the prefix `DominicSystem`. A custom venv for this project may also available to download from the repo. The venv has custom updated pyrpl files to allow it to work. You may need to create the fixes yourself.

3.3 Installing Dependencies

```
python -m venv venv
venv\Scripts\activate       # Windows
source venv/bin/activate    # Linux / Mac
```

Key packages:

Package	Version	Purpose
numpy	1.23.5	Arrays and numerical math
pandas	2.0.3	CSV reading, data merging, timestamps
matplotlib	3.7.5	Static PNG plot generation
scipy	1.10.1	Signal processing
pyrpl	0.9.7.0	Red Pitaya Lock-In Amplifier control via FPGA
potentiostat	0.0.4	Rodeostat USB communication
pyserial	3.5	Serial port communication
PyQt5	5.15.9	GUI framework required by PyRPL
plotly	any recent	Interactive HTML plots
paramiko	3.5.1	SSH utilities used by some RP libs

```
pip install numpy==1.23.5 pandas==2.0.3 matplotlib==3.7.5 scipy==1.10.1
pip install pyrpl==0.9.7.0 potentiostat==0.0.4 pyserial==3.5 PyQt5
   ==5.15.9
pip install plotly paramiko pyqtgraph==0.11.1
```

> **Note:** The full dependency list is in the README. PyRPL is finicky about Python version so be sure you have the good one.

4 Repository File Guide

The `DominicSystem` prefixed files are the main ones, cleaned up and ready to use. The `A` prefix versions are the same scripts with more comments and extra features for developing. A LLM was used too add to comments and descriptions of the functions to help new users navigate better. The others are older dev versions kept for reference and testing. Run `setup_scope.py` once to create `scope_config.yaml` if you don't download it with the repo.

4.1 DominicSystem Files — Use These

File	What it does
`DominicSystemAStartsystem.py`	Full experiment entry point (RP + Rodeostat). Launches all three subprocesses at once, merges the RP CSVs, generates 7-panel PNG and HTML plots, calls cycle averaging. Run this for a full ACCV run.

File	What it does
`DominicSystemAStartBoth-RedpitayasDCandLockIn.py`	Red Pitaya only, no Rodeostat. Same flow but skips the potentiostat subprocess. Use when driving the cell externally or triggering manually.
`DominicSystemALockIn-Amplifier.py`	Controls RP#1 as a software Lock-In via PyRPL. Generates 500 Hz AC, demodulates response, streams R and Theta to `lockin_results_*.csv`. Change frequency, amplitude, and decimation here.
`DominicSystemADCMonitor-WithACBiasRemover.py`	Controls RP#2 as a DC voltage monitor. Reads the actual DC potential at the potentiostat output and saves to `dc_voltage_*.csv`. Match decimation to the Lock-In.
`DominicSystemARodeostat.py`	Controls the Rodeostat over USB serial. Runs the DC voltage ramp, saves current and voltage to `rodeostat_data/`. The RP analysis doesn't use this CSV but it's saved for reference.
`DominicSystempostdataplot-create.py`	Post-processing. Splits data into CV cycles, averages R and Theta across them, generates cycle-averaged plots with $\pm 1\sigma$ shading. Runs automatically at end of acquisition.

4.2 A Files — Use These if DominicSystem is not working

File	What it does
`AStartsystem.py`	Full experiment entry point (RP + Rodeostat). Launches all three subprocesses at once, merges the RP CSVs, generates 7-panel PNG and HTML plots, calls cycle averaging. Run this for a full ACCV run.
`AStartBoth-RedpitayasDCandLockIn.py`	Red Pitaya only, no Rodeostat. Same flow but skips the potentiostat subprocess. Use when driving the cell externally or triggering manually.
`ALockInAmplifier.py`	Controls RP#1 as a software Lock-In via PyRPL. Generates 500 Hz AC, demodulates response, streams R and Theta to `lockin_results_*.csv`. Change frequency, amplitude, and decimation here.

File	What it does
`ADCMonitorWithACBiasRemover.py`	Controls RP#2 as a DC voltage monitor. Reads the actual DC potential at the potentiostat output and saves to `dc_voltage_*.csv`. Match decimation to the Lock-In.
`ARodeostat.py`	Controls the Rodeostat over USB serial. Runs the DC voltage ramp, saves current and voltage to `rodeostat_data/`. The RP analysis doesn't use this CSV but it's saved for reference.
`postdataplotcreate.py`	Post-processing. Splits data into CV cycles, averages R and Theta across them, generates cycle-averaged plots with $\pm 1\sigma$ shading. Runs automatically at end of acquisition.

4.3 Utility Files

File	What it does
`redpitaya_scpi(updated for py 3.8).py`	SCPI TCP wrapper used internally by the DC monitor script. Use this version, not the original.
`scope_config.yml`	PyRPL config loaded automatically by the Lock-In script. Don't delete it. Regenerate with `create_lockin_config.py` if needed.
`create_lockin_config.py`	Regenerates the PyRPL config. Run if you change the Lock-In frequency or channels.
`setup_scope.py`	Sets oscilloscope parameters and creates `scope_config.yaml`. Run once before first use.
`datacompareplotLOCKIN.py`	Plots Lock-In CSVs from multiple runs side by side.
`ExtraGoldStandard-ComparisonPloting`	Compares RP Lock-In results against Signal Recovery 7280 gold-standard data.

4.4 Old / Dev Scripts

These are older versions kept for reference, you don't need them for normal use: `SimpleDC.py`, `LOCK+AC.py`, `Red&Rodeo.py`, `RedCtrlPyRpl.py`, `RodeostatControl.py`, `startboth.py`, `Rodeostat Cyclic Voltammetry.py`.

5 Running the System

5.1 Before You Start

- Both RPs on and connected to the network and set with the correct names in Lock in and A Dc Scripts
- SCPI server running on both RPs
- Rodeostat plugged in, COM port set correctly
- Cell connected, Python 3.8 venv active

5.2 Set Your IP Addresses

Open `DominicSystemALockInAmplifier.py` and `DominicSystemADCMonitorWithACBiasRemover.py` and update the addresses:

```
rp = pyrpl.Pyrpl(config='scope_config', hostname='rp-xxxxxx.local')  #
   Lock-In RP
rp = redpitaya_scpi.scpi('rp-xxxxxx.local')                          #
   DC monitor RP
```

5.3 Running

With Rodeostat:

```
python DominicSystemAStartsystem.py
```

Red Pitayas only:

```
python DominicSystemAStartBothRedpitayasDCandLockIn.py
```

Press Enter once to launch. Wait for both RPs to connect (PyRPL takes 20–60 s first time). Press Enter again to start. Don't close the terminal. Plots open automatically when done.

5.4 Output Files

All saved to `test_data/`:

- `lockin_results_*.csv` — R, Theta, X, Y vs time
- `dc_voltage_*.csv` — DC voltage vs time
- `combined_results_*.csv` — merged RP data
- `accv_combined_*.png` / `.html` — 7-panel plots
- `accv_cycle_averaged_*.html` — cycle-averaged result (opens automatically)
- `rodeostat_data/` — Rodeostat CSV (full run only)

6 How It Works

Sync: A `start_time.txt` is written 2 seconds in the future. Each subprocess waits until that timestamp before starting, keeping both RPs aligned even if they connect at different speeds.

Lock-In (RP#1): PyRPL programs the FPGA as a digital Lock-In Amplifier. Generates a 500 Hz AC sine on the output and demodulates the response to give X, Y, R, and Theta continuously.

DC Monitor (RP#2): A DC mode over SCPI. Reads the actual voltage at the potentiostat output and streams it to CSV.

Rodeostat: Controlled via the `potentiostat` library over USB. Runs the triangular DC ramp across the electrochemical window.

Post-processing: Finds voltage turning points to segment data into individual cycles, then averages R and Theta across them to reduce noise.

7 Calibration

Run a dummy cell first (resistor or RC circuit) and compare R and Theta to what you'd expect before running real samples.

- **Rodeostat** matched Autolab PGSTAT302N results closely. Check current range and gain match your cell.
- **Lock-In RP** matched Signal Recovery 7280. Calibrate amplitude scale and phase offset with a known impedance using gain. check hardware jumpers for $\pm 1\,\mathrm{V}$ or $\pm 20\,\mathrm{V}$ input range and make sure the scale factor in the script is correct.
- **DC Monitor RP** Calibrate DC using known voltage using gain. check hardware jumpers for $\pm 1\,\mathrm{V}$ or $\pm 20\,\mathrm{V}$ input range and make sure the scale factor in the script is correct.

8 Troubleshooting

Problem	Fix
PyRPL won't connect	Check address, ping the board, check Ethernet cable.
PyRPL slow to connect	Normal , FPGA upload sometimes takes time on connect.
SCPI connection refused	SCPI server not started. Go to web interface and click Run.
Rodeostat not found	Wrong COM port. Check Device Manager and update the script.
No output CSV found	Something failed before data saved. Check terminal output for errors.
Poor time sync	Make sure both RPs fully connected before pressing Enter the second time. Make Sure Decimations are Close
No HTML plots	`pip install plotly`
PyRPL opens GUI window	Add `gui=False` to the `Pyrpl()` call.
Noisy or empty DC data	AC Filter in DCMonitor is to low or high, adjust to filter off AC but keep DC Ramp
Module not found	Wrong Python version or venv not active. Use Python 3.8.
Noisy Data or No shape	Possible Time Sync Mismatch adjust Decimation, Adjusted Bandwidthfilter on both devices.

If you find that your data is incredibly noisy or has no shape or usable data, try adjusting the decimation, bandwidth filter on lock in, filter on DC ramp, matching the run times, etc. You may need to mess with the filters a bit and that is okay and normal from experiment to experiment.

9 Quick-Start

1. Power on both RPs, connect Ethernet.
2. Start SCPI server on both via web interface.
3. Plug in Rodeostat, check COM port.
4. Activate venv
5. Run `python DominicSystemAStartsystem.py` (or the RP-only version).
6. Press Enter to launch. Wait for connections.
7. Press Enter again to start. Don't close the terminal.
8. Results open in browser automatically. CSVs in `test_data/`.

10 Experimental Results

10.1 Rodeostat vs. Autolab CV Comparison

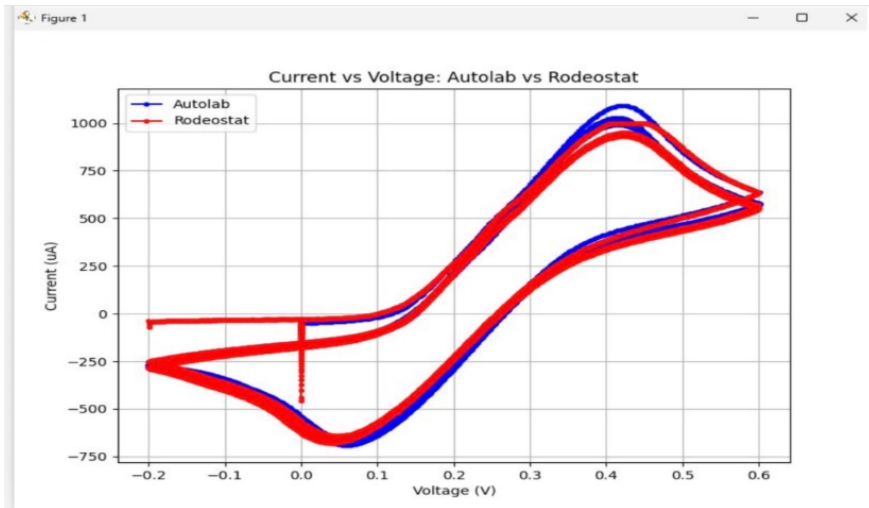


Figure 2: CV comparison between the Autolab PGSTAT302N (blue) and Rodeostat (red). The traces match closely across the full sweep. Generated by `DominicSystemARodeostat.py`.

10.2 Redpitaya LIA and DC Ramp Monitor vs. Signal Recovery Comparison

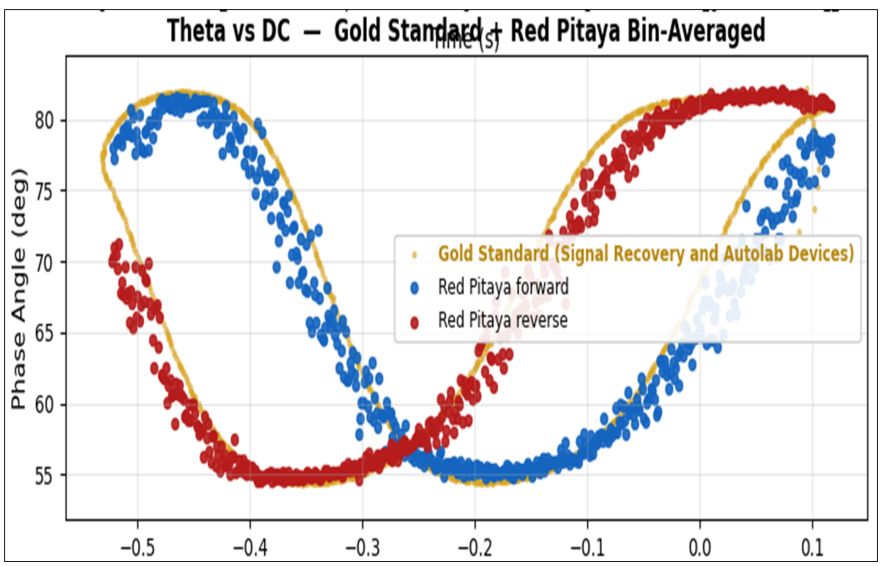


Figure 3: LIA & DC Monitor comparison between the Signal Recovery 7280 (gold) and Redpitayas (red). The traces match closely across the full sweep. Generated by `DominicSystemAStartBothRedpitayasDCandLockIn.py`.

10.3 Full ACCV Output — Dual Red Pitaya

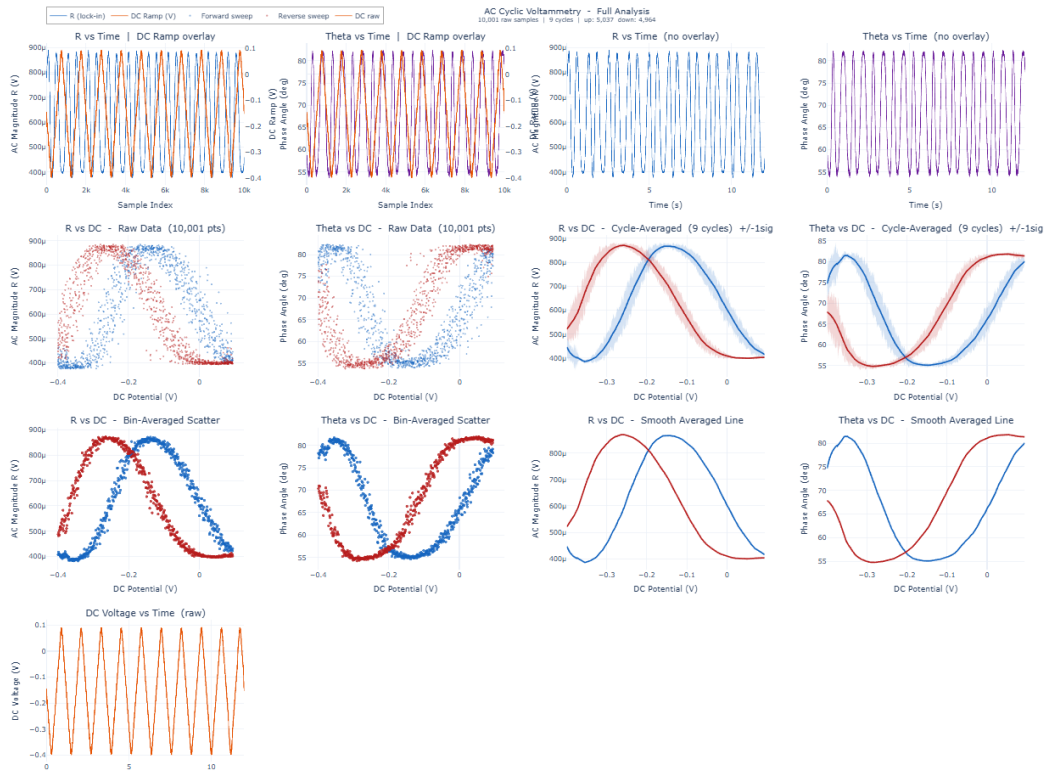


Figure 4: Full ACCV analysis (10,001 samples, 9 cycles). Top row: R and Theta vs time with DC ramp overlay. Middle row: raw scatter and cycle-averaged results with $\pm 1\sigma$ shading. Bottom row: bin-averaged and smooth-averaged line plots. Generated by `DominicSystemAStartsystem.py` and `DominicSystempostdataplotcreate.py`.

11 Known Limitations and Future Improvements

The current system achieves its core objectives replacing commercial potentiostat and lock-in instrumentation with open-source equivalents at comparable accuracy, but several hardware limitations affect signal quality and integration. The following describes the known weaknesses of the current design and the proposed directions for future hardware revisions.

11.1 Current Limitations

AC+DC Biasing PCB. The custom AC+DC biaser PCB injects the Red Pitaya AC signal onto the Rodeostat working electrode externally, outside the potentiostat's control loop. Because the AC is added passively rather than being part of the feedback path, the Rodeostat does not actively regulate or compensate for the AC component. This can introduce distortion and coupling artifacts, particularly at higher amplitudes or frequencies, and the quality of the bias injection is sensitive to impedance matching between the PCB and the cell. This can distort data widely and mess with accuracy.

Rodeostat TIA. The Rodeostat's onboard Transimpedance Amplifier (TIA) was designed for general purpose use and is not optimized for the noise floor and bandwidth requirements of AC CV

experiments. At the AC frequencies used in this system (500 Hz and above), the TIA can contribute noise that limits the clean recovery of R and Θ, particularly at low current ranges.

Lock-In Plot Cleanliness. The Red Pitaya Lock-In data, while accurate, can exhibit cycle-to-cycle scatter that requires post-processing averaging to produce clean ACCV plots. The raw R vs. V_{DC} traces often need bin-averaging or smoothing before features are visible. This is partly a consequence of the external AC biasing not being inside the control loop, and partly a result of TIA noise feeding into the Lock-In input. The two redpitayas also do not share a clock and are not daisy chained which can cause timing errors in measurements causing data to become very blocky and hard to read as the data shifts left to right.

12 Hardware Specifications

Specification	**Rodeostat**	**Red Pitaya STEMlab 125-14**
Type	Open-source potentiostat	FPGA-based data acquisition and signal generation platform
Output voltage range	±1, ±2, ±5, and ±10 V, software select	±1 V into high impedance load
Input voltage range	-	±1 V or ±20 V, selected by hardware jumper
AC signal generation	-	Arbitrary waveform, approximately 0 Hz to 60 MHz
AC frequency used in this project	-	500 Hz, tunable in software
FPGA clock frequency	-	125 MHz internal reference clock
ADC sample rate (hardware maximum)	≈200 Hz	125 MS/s
ADC sample rate (this project)	≈200 Hz	≈1 MS/s via PyRPL decimation
ADC resolution	16-bit	14-bit
DAC resolution	12-bit	14-bit
TIA built-in	Yes	-
Current measurement ranges	±1, ±10, ±100, and ±1000 μA, software select	-
Minimum detectable current	Approximately 1 μA on the lowest range	-

Specification	Rodeostat	Red Pitaya STEMlab 125-14
Minimum resolvable voltage	-	≈122 μV on the ±1 V range (14-bit ADC); ≈2.4 mV on the ±20 V range
Onboard processor	ARM Cortex-M4, Adafruit ItsyBitsy M4 Express, 120 MHz	Dual-core ARM Cortex-A9 with Xilinx Zynq 7010 FPGA

12.1 Future Improvements

Integrated External V_{in} on the Rodeostat PCB. The most important improvement would be to add a dedicated external voltage input (V_{in}) directly to the Rodeostat PCB, modeled on the External V_{in} present on instruments like the Autolab PGSTAT series. In the Autolab, the external V_{in} feeds directly into the potentiostat's control loop, meaning the AC signal becomes part of the regulated output rather than being injected passively after the fact. Adding this to the Rodeostat would allow the Red Pitaya AC signal to be summed cleanly inside the feedback path, eliminating the external biaser PCB entirely and significantly improving the clarity of the AC waveform applied to the cell. Similarly a new custom potentiostat could be designed to simply generate the ACDC Wave on its own using a better chip.

Integrated TIA Redesign. A redesigned TIA optimised for AC CV would improve the noise floor and gain scaling at the Lock-In input and reduce the post processing.

Single Custom ASIC or Integrated Board. The idea is a single integrated board that combines both the AC signal generation and the DC potentiostat control in one closed loop system with a nicer chip such as the AD5941 which can do AC DC Biasing on its own. Such a design would generate both the DC ramp and the AC excitation natively, with the AC summed inside the control loop from the start. An external V_{in} would still be there for flexibility, but the default operation would not require any external Red Pitaya signal injection at all. The TIA would be co-designed with the redpitaya's to have the entirety of the project run on a single clock to remove time drift, and board for the best accuracy, precision, and simplicity. The redpitayas could also be attached to the PCB to read the signal directly without extra components to make the design more simple and clean.

13 Acknowledgments

Work conducted in the Saraf Lab, University of Nebraska–Lincoln.

Funding: Vajra Instruments and the National Institutes of Health (NIH).

Special Thanks to Joydip Dey and Khirul Titu and Tawfiq Anik and Lihn Hua (Saraf Lab) for help with electrochemistry, sample creation, experiment interpretation,design, and being good mentors, and to Dr. Ravi Saraf for guidance and supporting and funding this project.